DAY 3 — OBJECT-ORIENTED PROGRAMMING (OOP)

Completed Java Tasks • Concepts, Programs, and Sample Output

1. Fundamental Concepts

Classes and Objects

A class is a blueprint that defines data and behaviour. An object is an instance of a class. For example, Student is a class and one particular student is an object.

Constructors

A constructor initializes an object. Its name is the same as the class name and it has no return type.

Encapsulation

Encapsulation combines data and methods in a class and protects data using access control, commonly private fields and public getters/setters.

Inheritance

Inheritance allows a child class to reuse or extend the features of a parent class using the extends keyword.

Polymorphism

Polymorphism means one name or interface can have different behaviours. Overloading is compile-time polymorphism; overriding is runtime polymorphism.

Abstraction

Abstraction shows essential features and hides implementation details. Java supports it through abstract classes and interfaces.

Access Modifiers

public: accessible from anywhere; private: within its class only; protected: within the package and subclasses; default (no keyword): within the same package.

Methods and Method Overloading

A method defines an action. Overloading means using the same method name with different parameter lists. Changing only the return type is not enough.

2. Task 1 — Create Student and Employee Classes

This program creates both classes, adds properties and methods, and demonstrates constructors and encapsulation.

```
class Student {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public int getAge() { return age; }

    public void setAge(int age) {
        if (age > 0) this.age = age;
    }

    public void display() {
        System.out.println("Student: " + name + ", Age: " + age);
    }
}

class Employee {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public double getSalary() { return salary; }
```

```
    public void display() {
        System.out.println("Employee: " + name + ", Salary: " + salary);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Aarav", 20);
        Employee e = new Employee("Neha", 35000);

        s.display();
        e.display();
        s.setAge(21);
        System.out.println("Updated age: " + s.getAge());
    }
}
```

Sample Output

```
Student: Aarav, Age: 20\nEmployee: Neha, Salary: 35000.0\nUpdated age: 21
```

3. Task 2 — Demonstrate Constructors, Inheritance, and Encapsulation

Person is the parent class and Student is the child class. Private data and a getter demonstrate encapsulation; constructors initialize objects; extends demonstrates inheritance.

```
class Person {
    private String name;

    public Person(String name) { this.name = name; }
    public String getName() { return name; }
}

class Student extends Person {
    private int rollNo;

    public Student(String name, int rollNo) {
        super(name);
        this.rollNo = rollNo;
    }

    public void display() {
        System.out.println("Name: " + getName());
        System.out.println("Roll Number: " + rollNo);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Aarav", 101);
        s.display();
    }
}
```

Sample Output

```
Name: Aarav\nRoll Number: 101
```

4. Task 3 — Console-Based Student Management Application

This menu-driven application lets the user add students, display all records, and search by roll number. Records are stored in an ArrayList while the program runs.

```
import java.util.ArrayList;
import java.util.Scanner;

class Student {
    private int rollNo;
    private String name;
    private int age;

    public Student(int rollNo, String name, int age) {
        this.rollNo = rollNo;
        this.name = name;
        this.age = age;
    }

    public int getRollNo() { return rollNo; }

    public void display() {
        System.out.println("Roll No: " + rollNo
            + ", Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Student> students = new ArrayList<>();
        int choice;

        do {
            System.out.println("\n--- Student Management ---");
            System.out.println("1. Add Student");
            System.out.println("2. Display All Students");
            System.out.println("3. Search by Roll Number");
            System.out.println("0. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1:
                    System.out.print("Roll number: ");
                    int roll = sc.nextInt();
                    sc.nextLine();
                    System.out.print("Name: ");
                    String name = sc.nextLine();
                    System.out.print("Age: ");
                    int age = sc.nextInt();
                    students.add(new Student(roll, name, age));
                    System.out.println("Student added.");
                    break;
                case 2:
                    if (students.isEmpty()) {
                        System.out.println("No students found.");
                    } else {
                        for (Student s : students) s.display();
                    }
                    break;
                case 3:
                    System.out.print("Enter roll number to search: ");
                    int target = sc.nextInt();
                    boolean found = false;
                    for (Student s : students) {
                        if (s.getRollNo() == target) {
                            s.display();
                            found = true;
                            break;
                        }
                    }
                    if (!found) System.out.println("Student not found.");
                    break;
                case 0:
                    System.out.println("Exiting application.");
                    break;
                default:
                    System.out.println("Invalid choice.");
            }
        } while (choice != 0);
        sc.close();
    }
}
```

5. Example Run

```
--- Student Management ---
```

```
1. Add Student
2. Display All Students
3. Search by Roll Number
0. Exit
Enter choice: 1
Roll number: 101
Name: Aarav Tripathi
Age: 20
Student added.

--- Student Management ---
1. Add Student
2. Display All Students
3. Search by Roll Number
0. Exit
Enter choice: 2
Roll No: 101, Name: Aarav Tripathi, Age: 20

--- Student Management ---
1. Add Student
2. Display All Students
3. Search by Roll Number
0. Exit
Enter choice: 0
Exiting application.
```

6. Compile and Run

Save one program at a time as Main.java. Open a terminal in that folder and run: javac Main.java, then java Main. The examples each use a public class named Main, so compile them separately.

Note: The Student Management Application stores data in memory only. Records are lost when the program exits; database/file storage is not included in the given task.